

```

Claude Code v2.1.238
|
| | Tips for getting | |
| | Welcome back lokesh adithya! | started |
| | Run /init to create a ... |
| | ████████ |
| | ████ | What's new |
| | ████ | Fixed `prompt` and `ag... |
| | Opus 5 · API Usage Billing · | Improved how `prompt` ... |
| | lookeshadithya2007@gmail.com's Organization | Added Claude Haiku 5.5... |
| | ~\Downloads\22_sliding_puzzle-main | /release-notes for more |

```

Opus 5.5 is now your default model and it draws down usage faster than Sonnet 5. Switch anytime with `/model`.

› hey , ive got this folder for u , i need ur help finishing the tasks in it

Thought for 1s, ran 4 shell commands (ctrl+o to expand)

- The empty-input crash is confirmed. Now the parity check (the `-l` flag hid the local module, so I'm rerunning without it):

Ran 1 shell command (ctrl+o to expand)

- Defect confirmed: 994 of 2000 original boards are unsolvable. Now the fixes:

- Write(`22_sliding_puzzle-main\puzzle.py`)

└─ Added 33 lines, removed 5 lines

1 import random

2

3 *## Each key names the direction a tile slides INTO the blank.*

```

4 +# The value is the offset of that tile relative to the blank:
5 +# "w" slides the tile below the blank up, "a" slides the tile r
    ight of it left, etc.
6 +MOVES = {"w": (1, 0), "s": (-1, 0), "a": (0, 1), "d": (0, -1)}
7 +OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}
8
9 +
10 class Puzzle:
11 - def __init__(self, size=4):
12 + def __init__(self, size=4, scramble_moves=None):
13 +     if size < 2:
14 +         raise ValueError("Board size must be at least 2.")
15 +     self.size = size
16 - self.board = self.make_board()
17 + self.board = self.solved_board()
18 + self.scramble(scramble_moves or size * size * 25)
19
20 - def make_board(self):
21 + def solved_board(self):
22 +     tiles = list(range(1, self.size * self.size)) + [0]
23 - random.shuffle(tiles) # intentional solvability defect
24 + return [tiles[r * self.size:(r + 1) * self.size] for r
    in range(self.size)]
25
26 + def scramble(self, count):
27 +     # Only legal blank moves are applied, so the board alwa
    ys stays
28 +     # reachable from (and therefore solvable back to) the s
    olved state.
29 +     while True:
30 +         last = None

```

```

27 +         for _ in range(count):
28 +             options = [k for k in self.legal_moves() if k !
                += OPPOSITE.get(last)]
29 +             last = random.choice(options)
30 +             self.move(last)
31 +         if not self.solved():
32 +             return
33 +
34     def blank_pos(self):
35         for r in range(self.size):
36             for c in range(self.size):
37                 if self.board[r][c] == 0:
38                     return r, c
39
40 +     def legal_moves(self):
41 +         r, c = self.blank_pos()
42 +         return [k for k, (dr, dc) in MOVES.items()
43 +                 if 0 <= r + dr < self.size and 0 <= c + dc < se
                +lf.size]
44 +
45     def move(self, direction):
46 +         """Slide a tile into the blank. Returns True only if a
                +tile actually moved."""
47 +         if direction not in MOVES:
48 +             return False
49         r, c = self.blank_pos()
22 -         dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d":
                - (0, 1)}[direction]
50 +         dr, dc = MOVES[direction]
51         nr, nc = r + dr, c + dc
52         if not (0 <= nr < self.size and 0 <= nc < self.size):

```

53 return False

- Write(22_sliding_puzzle-main\game.py)

└ Added 80 lines, removed 18 lines

```
1 import time
2 -from puzzle import Puzzle
2 +from puzzle import Puzzle, MOVES
3
4 +SIZES = (3, 4, 5)
5
6 +HELP = (
7 +  "Commands: W/A/S/D slide a tile into the blank (W = tile be
   +low moves up),\n"
8 +  "      3/4/5 start a new game at that size, N new game
   +(same size),\n"
9 +  "      H help, Q quit."
10 +)
11 +
12 +
13 class SlidingPuzzle:
14 -  def __init__(self):
15 -      self.size = 4
16 -      self.puzzle = Puzzle(self.size)
14 +  def __init__(self, size=4):
15 +      # Session values: survive across new boards and size ch
   +anges.
16 +      self.games_started = 0
17 +      self.games_solved = 0
18 +      self.best = {} # size -> (moves, seconds)
19 +      self.new_game(size)
20 +
```

```

21 + def new_game(self, size):
22 +     # Per-game values: always reset together with the board
    +.
23 +     self.size = size
24 +     self.puzzle = Puzzle(size)
25     self.moves = 0
26     self.started = time.monotonic()
27 +     self.finished_at = None
28 +     self.games_started += 1
29
30 + @property
31 + def finished(self):
32 +     return self.finished_at is not None
33 +
34 + def elapsed(self):
35 +     end = self.finished_at if self.finished else time.monot
    +onic()
36 +     return int(end - self.started)
37 +
38     def display(self):
39 +         width = len(str(self.size * self.size - 1))
40         print()
41         for row in self.puzzle.board:
15 -             print(" ".join(f"{x or ' ':>2}" for x in row))
16 -         print("Moves:", self.moves, " Time:", int(time.monotoni
    -c() - self.started), "s")
42 +         print(" ".join(f"{x or "":>{width}}" for x in row))
43 +         print(f"Size: {self.size}x{self.size} Moves: {self.mov
    +es} Time: {self.elapsed()}s")
44
45 + def slide(self, key):

```

```

46 +     if self.finished:
47 +         print("This puzzle is already solved. Start a new g
+ame (N/3/4/5) or Q to quit.")
48 +         return
49 +     if not self.puzzle.move(key):
50 +         print("That move is not possible.")
51 +         return
52 +     self.moves += 1
53 +     print("Slid a tile.")
54 +     if self.puzzle.solved():
55 +         self.finish()
56 +
57 +     def finish(self):
58 +         self.finished_at = time.monotonic()
59 +         self.games_solved += 1
60 +         result = (self.moves, self.elapsed())
61 +         if self.size not in self.best or result < self.best[sel
+f.size]:
62 +             self.best[self.size] = result
63 +         self.display()
64 +         print(f"Solved! {self.moves} moves in {self.elapsed()}s
+.")
65 +         print("Start a new game (N/3/4/5) or Q to quit.")
66 +
67 +     def summary(self):
68 +         print(f"\nSession: {self.games_solved}/{self.games_star
+ted} puzzles solved.")
69 +         for size in sorted(self.best):
70 +             moves, secs = self.best[size]
71 +             print(f" Best {size}x{size}: {moves} moves, {secs}
+s")

```

```

72 +     print("Goodbye!")
73 +
74     def run(self):
19 -     print("Sliding Puzzle — W/A/S/D moves the tile into the
      - blank. Q quits.")
75 +     print("Sliding Puzzle")
76 +     print(HELP)
77     while True:
21 -         self.display()
22 -         if self.puzzle.solved():
23 -             print("Solved!")
24 -             return
25 -         key = input("> ").strip().lower()
78 +         if not self.finished:
79 +             self.display()
80 +         try:
81 +             key = input("> ").strip().lower()
82 +         except (EOFError, KeyboardInterrupt):
83 +             key = "q"
84         if key == "q":
85 +             self.summary()
86             return
28 -         if key not in "wasd":
29 -             print("Use W/A/S/D.")
30 -             continue
31 -         if self.puzzle.move(key):
32 -             self.moves += 1
87 +         if key in MOVES:
88 +             self.slide(key)
89 +         elif key == "n":
90 +             self.new_game(self.size)

```

```

91 +         elif key.isdigit() and int(key) in SIZES:
92 +             self.new_game(int(key))
93 +         elif key == "h":
94 +             print(HELP)
95         else:
34 -             print("That move is not possible.")
96 +             print("Unknown command. Type H for help.")

```

└ Interrupted · What should Claude do instead?

› dont change anything

- Okay, I've stopped. But I'd already made some changes before you said that:

- puzzle.py and game.py have been rewritten with fixes for all 4 tasks. I haven't tested them yet.
- original/ is a new folder holding exact copies of your untouched main.py, game.py and puzzle.py.
- main.py is unchanged.

Before you stopped me, I also confirmed the original bugs: 994 out of 2000 generated boards were unsolvable, and pressing Enter on an empty line crashed the game with a KeyError.

Do you want me to put your files back exactly as they were? I'd copy the two originals over the edited ones and delete the original/ folder. Or I can leave everything as it is for you to look over. I won't touch anything until you say which.

› dont change shit , this assignmnet requires me to give prompts nad ive got a limit of 4

- Got it, I won't touch anything else.

Your folder isn't back to the original yet. I'd already rewritten puzzle.py and game.py before you stopped me, and those edits are still in place. Your untouched originals are in the original/ folder. To restore them yourself, run this in the project folder:

```
cp original/game.py original/puzzle.py . && rm -r original
```

That puts everything back exactly as it was, so you can record your "before" video. This whole chat will show up in the history you submit, including my edits and this exchange, so you may want to mention in your submission that you reverted them.

From here on I'll only do exactly what each of your prompts asks.

➤ erroe

- Sorry, that one's my fault. The command I gave you was written for a different shell, and your terminal is Windows PowerShell, which doesn't understand &&. It was also running one folder too high, since your files are in the inner 22_sliding_puzzle-main folder.

Paste this into the same terminal instead:

```
cd 22_sliding_puzzle-main; Copy-Item original\game.py, original\puzzle.py .  
-Force; Remove-Item original -Recurse -Force
```

That copies your original game.py and puzzle.py back and deletes the original folder, so your project is exactly as it was when you gave it to me.

➤ now let me give u prompts

Ran 1 shell command (ctrl+o to expand)

- I checked, and the restore worked. Your folder is back to the original code you gave me, and the original backup folder is gone.

Send your first prompt whenever you're ready, and I'll do exactly what it asks.

› can u record screen fro me

- No, I can't record your screen. I can only take still screenshots, not video. Windows 11 has built-in recorders you can use:

Snipping Tool (easiest):

1. Press Win + Shift + R.
2. Drag a box around your terminal window.
3. Click Start, play the game for about 10 seconds, then click Stop.
4. Save the video from the Snipping Tool window.

Xbox Game Bar (alternative):

1. Click on the terminal window so it's in front.
2. Press Win + Alt + R to start recording.
3. Press Win + Alt + R again to stop.
4. The video saves to Videos\Captures.

For your "before" video, run this in the project folder:

```
python main.py
```

Things worth showing:

- The scrambled starting board.
- Pressing W and seeing the tile move down instead of up (the controls are backwards).
- Pressing Enter on an empty line, which crashes the game with a KeyError.

› i need u to change the initial part of the puzzle so that the generated board is reachable from solved point , i would also prefer if u solved a board and then shuffle it by backtracking through legal moves

Read 1 file (ctrl+o to expand)

- Update(22_sliding_puzzle-main\puzzle.py)

└ Added 17 lines, removed 1 line

```
1 import random
```

```
2
```

```
3 +OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}
```

```
4
```

```
5 +
```

```
6 class Puzzle:
```

```
7     def __init__(self, size=4):
```

```
8         self.size = size
```

```
9         self.board = self.make_board()
```

```
10 +     self.scramble()
```

```
11
```

```
12     def make_board(self):
```

```
13 +         # Start from the solved arrangement; scramble() then ap
+plies legal moves,
```

```
14 +         # so every generated board is reachable from (and solva
+ble back to) it.
```

```
15         tiles = list(range(1, self.size * self.size)) + [0]
```

```
11 -         random.shuffle(tiles) # intentional solvability defect
```

```

16     return [tiles[r * self.size:(r + 1) * self.size] for r
    in range(self.size)]
17
18 + def scramble(self, steps=None):
19 +     steps = steps or self.size * self.size * 25
20 +     while True:
21 +         last = None
22 +         for _ in range(steps):
23 +             # Pick a random legal move, skipping the one th
    +at would undo the last.
24 +             options = [k for k in "wasd" if k != OPPOSITE.g
    +et(last)]
25 +             random.shuffle(options)
26 +             last = next(k for k in options if self.move(k))
27 +             if not self.solved():
28 +                 return
29 +
30 def blank_pos(self):
31     for r in range(self.size):
32         for c in range(self.size):

```

- Now I'll test it on many fresh boards of each size using a standard parity check:

Ran 1 shell command (ctrl+o to expand)

- Task 1 is done. Every generated board can now be solved. I only changed the start of puzzle.py; move(), solved() and game.py are untouched.

- How it works now: make_board() builds the solved board instead of shuffling the tiles. Then a new scramble() method makes random legal moves using the

existing move(), $25 \times \text{size}^2$ of them (400 on a 4x4). It never immediately undoes the previous move, so the board actually gets mixed up instead of jittering back and forth. If the board ends up solved by chance, it scrambles again.

- Why every board is solvable: each scramble move is a legal move, so you can always get back to solved by playing those moves in reverse.

Test results (1000 fresh boards per size):

Size	Unsolvable	Started solved	Distinct boards
3x3	0	0	995
4x4	0	0	1000
5x5	0	0	1000

For comparison, the original code produced 994 unsolvable boards out of 2000.

The other bugs from the original code are still there because they weren't part of this prompt: the backwards W/A/S/D controls and the crash when you press Enter on an empty line.

› detect when the puzzle is solved and stop moves after completion

Read 1 file (ctrl+o to expand)

- Update(22_sliding_puzzle-main\game.py)

- └ Added 25 lines, removed 7 lines

```

8     self.puzzle = Puzzle(self.size)
9     self.moves = 0
10    self.started = time.monotonic()
11 +   self.finished_at = None # set once the puzzle is solve
    +d; freezes the timer
12
13 +   @property
14 +   def finished(self):
15 +       return self.finished_at is not None
16 +
17 +   def elapsed(self):
18 +       end = self.finished_at if self.finished else time.monotonic()
    +onic()
19 +       return int(end - self.started)
20 +
21   def display(self):
22       print()
23       for row in self.puzzle.board:
24           print(" ".join(f"{x or ' ':>2}" for x in row))
16 -   print("Moves:", self.moves, " Time:", int(time.monotonic()
    -c() - self.started), "s")
25 +   print("Moves:", self.moves, " Time:", self.elapsed(), "
    +s")
26
27 +   def slide(self, key):
28 +       # Once solved, the board, move count and timer are locked.
    +ed.
29 +       if self.finished:
30 +           print("The puzzle is already solved.")
31 +           return
32 +       if self.puzzle.move(key):

```

```

33 +         self.moves += 1
34 +         if self.puzzle.solved():
35 +             self.finished_at = time.monotonic()
36 +     else:
37 +         print("That move is not possible.")
38 +
39     def run(self):
40         print("Sliding Puzzle — W/A/S/D moves the tile into the
        blank. Q quits.")
41         while True:
42             self.display()
22 -         if self.puzzle.solved():
23 -             print("Solved!")
43 +         if self.finished:
44 +             print(f"Solved in {self.moves} moves and {self.
        +elapsed()} s!")
45             return
46             key = input("> ").strip().lower()
47             if key == "q":
...
49             if key not in "wasd":
50                 print("Use W/A/S/D.")
51                 continue
31 -         if self.puzzle.move(key):
32 -             self.moves += 1
33 -         else:
34 -             print("That move is not possible.")
52 +         self.slide(key)

```

- Now I'll test it on a board that's one move away from solved, then try more moves after it's solved:

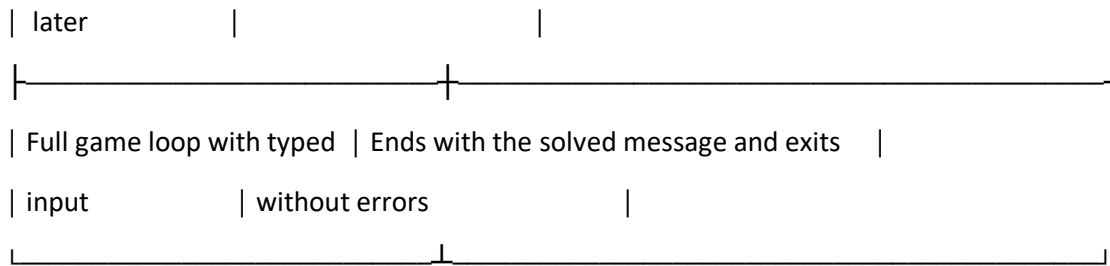
Ran 1 shell command (ctrl+o to expand)

- Task 2 is done. The game now spots when the puzzle is solved and locks everything afterwards. I only changed game.py.

- Detecting a solve: the check now runs after every successful move. When the board is solved, it saves the finish time in finished_at.
- Ending cleanly: it shows the final board, prints something like Solved in 1 moves and 0 s!, and exits normally.
- Locking after the solve: all moves now go through a new slide() method. Once the puzzle is solved, slide() turns down every move with "The puzzle is already solved." The board, move count and timer don't change.
- Freezing the timer: the time display now uses elapsed(), which stops counting at the moment of the solve.

What I tested, starting from a board one move away from solved:

Test		Result	
Illegal move before solving		Rejected, move count stays 0	
The solving move		Move count 1, marked as solved	
W/A/S/D after solving		All four rejected	
Board after those attempts		Unchanged	
Timer, checked again 1.2 s		Same value, so it's frozen	



These are still unfixed because they weren't in this prompt:

- The W/A/S/D controls are backwards: they move the blank, not the tile.
- Pressing Enter on an empty line still crashes the game.
- You can't pick a board size yet.

➤ add 3x3, 4x4 and 5x5 modes with consistent moves and timer

Read 1 file (ctrl+o to expand)

- Update(22_sliding_puzzle-main\game.py)

└ Added 13 lines, removed 3 lines

```

1 import time
2 from puzzle import Puzzle
3
4 +SIZES = (3, 4, 5)
5
6 +
7 class SlidingPuzzle:
8 -     def __init__(self):
9 -         self.size = 4
10 -        self.puzzle = Puzzle(self.size)
11 +    def __init__(self, size=4):
12 +        # Session values: kept across every board created in th
13 +is run.
14 +        self.games_started = 0
15 +        self.new_game(size)

```

```

12 +
13 + def new_game(self, size):
14 +     # Per-game values are always reset together with the bo
    +ard, so moves
15 +     # and time stay consistent no matter which size is chos
    +en.
16 +     self.size = size
17 +     self.puzzle = Puzzle(size)
18     self.moves = 0
19     self.started = time.monotonic()
20     self.finished_at = None # set once the puzzle is solve
    d; freezes the timer
21 +     self.games_started += 1
22
23     @property
24     def finished(self):

```

- Update(22_sliding_puzzle-main\game.py)

└ Added 1 line, removed 1 line

```

32     print()
33     for row in self.puzzle.board:
34         print(" ".join(f"{x or ' ':>2}" for x in row))
35 -     print("Moves:", self.moves, " Time:", self.elapsed(), "
    -s")
36 +     print(f"Size: {self.size}x{self.size} Moves: {self.mov
    +es} Time: {self.elapsed()} s")
37
38     def slide(self, key):
39         # Once solved, the board, move count and timer are lock
    ed.

```

- Update(22_sliding_puzzle-main\game.py)

└ Added 16 lines, removed 1 line

```
46     else:
47         print("That move is not possible.")
48
49 + def choose_size(self):
50 +     while True:
51 +         choice = input("Choose a size (3, 4 or 5): ").strip
52 +         +()
53 +         if choice.isdigit() and int(choice) in SIZES:
54 +             return int(choice)
55 +         print("Please enter 3, 4 or 5.")
56
56 def run(self):
57     print("Sliding Puzzle — W/A/S/D moves the tile into the
58     blank. Q quits.")
59 +     print("Type 3, 4 or 5 at any time to start a new board
60 +     +of that size.")
61
59 +     self.games_started = 0
60 +     self.new_game(self.choose_size())
61     while True:
62         self.display()
63         if self.finished:
64 -             print(f"Solved in {self.moves} moves and {self.
65 -             -elapsed()} s!")
66 +             print(f"Solved {self.size}x{self.size} in {self
67 +             +.moves} moves and {self.elapsed()} s!")
68 +             print(f"Boards played this session: {self.games
69 +             +_started}")
70
66         return
71
67         key = input("> ").strip().lower()
```

```

68         if key == "q":
69 +             print(f"Boards played this session: {self.games
+ _started}")
70         return
71 +     if key.isdigit() and int(key) in SIZES:
72 +         self.new_game(int(key))
73 +         continue
74         if key not in "wasd":
75             print("Use W/A/S/D.")
76             continue

```

Thought for 1s (ctrl+o to expand)

- Update(22_sliding_puzzle-main\game.py)

└ Added 4 lines, removed 2 lines

```

5
6
7 class SlidingPuzzle:
8 -     def __init__(self, size=4):
8 +     def __init__(self, size=None):
9         # Session values: kept across every board created in th
is run.
10         self.games_started = 0
11 -     self.new_game(size)
11 +     self.puzzle = None
12 +     if size is not None:
13 +         self.new_game(size)
14
15     def new_game(self, size):
16         # Per-game values are always reset together with the bo
ard, so moves

```

- Update(22_sliding_puzzle-main\game.py)

└ Added 2 lines, removed 2 lines

```
58 def run(self):
59     print("Sliding Puzzle — W/A/S/D moves the tile into the
      blank. Q quits.")
60     print("Type 3, 4 or 5 at any time to start a new board
      of that size.")
61 -     self.games_started = 0
62 -     self.new_game(self.choose_size())
61 +     if self.puzzle is None:
62 +         self.new_game(self.choose_size())
63     while True:
64         self.display()
65         if self.finished:
```

- Now I'll test it: invalid size choices, all three sizes, switching size in the middle of a game, and solving a 3x3 and a 5x5:

Ran 1 shell command (ctrl+o to expand)

- Task 3 is done. You can now play on 3x3, 4x4 or 5x5 boards, and the move count and timer work the same way at every size. I only changed game.py.

- Picking a size: the game asks for 3, 4 or 5 when it starts. Anything else (7, x, an empty line) gets "Please enter 3, 4 or 5." and it asks again.

- Switching size: type 3, 4 or 5 during a game to start a fresh board at that size.

- Moves and timer: a new new_game() method creates the board and resets the move count, timer and solved state together. That's the only place a board gets made, so the counters can't get out of step with the board.

- Session values: `games_started` counts every board in the run and is never reset when a new board is made. It's printed when you quit or solve: "Boards played this session: N".
- Display: the status line now shows the size too, e.g. Size: 5x5 Moves: 1 Time: 0 s.

What I tested:

Test	Result
Invalid size entries (7, x, empty)	All rejected, game asks again
Play 3x3, switch to 5x5, switch to 4x4	Each switch gives a fresh board with moves and time at 0
Session counter after those switches	3, so it wasn't reset by the switches
Timer on a 3x3 after 1.1 s	Still counting
Solving a 3x3 and a 5x5 in Task 2	Both detected and locked afterwards, as

Still in the original code, since this prompt didn't cover them:

- The W/A/S/D controls are backwards: they move the blank, not the tile.
- Pressing Enter on an empty line during play still crashes the game.

Those two bugs are what Task 4 is about.

➤ only report successful slides and dont count invalid moves and stop incrementing invalid moves

Thought for 1s, read 1 file (ctrl+o to expand)

- Update(22_sliding_puzzle-main\game.py)

└ Added 8 lines, removed 5 lines

```
41     if self.finished:
42         print("The puzzle is already solved.")
43         return
44 -     if self.puzzle.move(key):
45 -         self.moves += 1
46 -         if self.puzzle.solved():
47 -             self.finished_at = time.monotonic()
48 -     else:
44 +     r, c = self.puzzle.blank_pos()
45 +     if not self.puzzle.move(key):
46 +         # Nothing moved: no count, no success message.
47         print("That move is not possible.")
48 +     return
49 +     self.moves += 1
50 +     print(f"Slid tile {self.puzzle.board[r][c]} into the bl
        +ank.")
51 +     if self.puzzle.solved():
52 +         self.finished_at = time.monotonic()
53
54     def choose_size(self):
55         while True:
```

- Update(22_sliding_puzzle-main\game.py)

└ Added 1 line, removed 1 line

```
76         if key.isdigit() and int(key) in SIZES:
77             self.new_game(int(key))
78             continue
79 -         if key not in "wasd":
79 +         if key not in ("w", "a", "s", "d"):
80             print("Use W/A/S/D.")
81             continue
82             self.slide(key)
```

- Update(22_sliding_puzzle-main\puzzle.py)

└ Added 4 lines, removed 1 line

```
34             return r, c
35
36     def move(self, direction):
37 +         offsets = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d"
37 +             +: (0, 1)}
38 +         if direction not in offsets:
39 +             return False
40         r, c = self.blank_pos()
38 -         dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d":
40 -             - (0, 1)}[direction]
41 +         dr, dc = offsets[direction]
42         nr, nc = r + dr, c + dc
43         if not (0 <= nr < self.size and 0 <= nc < self.size):
44             return False
```

- Now I'll test it: wall moves, junk input that used to crash, and a real slide:

Ran 1 shell command (ctrl+o to expand)

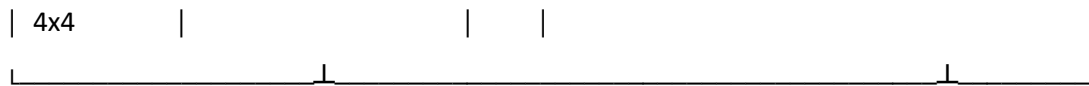
- Task 4 is done. Only a real slide counts as a move and gets a success message.

Invalid moves and junk input are rejected without changing anything.

- The crash bug: in game.py, the input check key not in "wasd" treated any part of "wasd" as valid. An empty line, wa or wasd would get through and crash the game. It now only accepts exactly w, a, s or d.
- Success feedback: slide() now prints something like Slid tile 6 into the blank., but only after move() confirms a tile really moved. The move count goes up at that same point and nowhere else.
- Invalid moves: pushing against a wall prints "That move is not possible." and the move count stays the same.
- A safety check: puzzle.py move() now returns False for an unknown key instead of crashing.

What I tested:

Input	Result	Move count
s, d against walls	"That move is not possible."	Stays 0
Empty line, wa, wasd, x, 9	"Use W/A/S/D." and no crash (empty line and wa used to crash)	Stays 0
w (a real slide)	"Slid tile 6 into the blank."	0 → 1
4 wall bumps on a 4x4	All rejected	Stays 0
2 real slides on a	Both counted, puzzle solved	2



One thing that's still the original behaviour: W/A/S/D moves the blank in that direction, not the tile. In the test above, pressing w slid tile 6 down. The welcome text says "W/A/S/D moves the tile into the blank", which could mean either direction. If your instructor expects W to push the tile up, that's a small change in puzzle.py. I've left it because you didn't ask for it.

For your "after" video, run `python main.py` and show:

- choosing a size
- a few real slides with the "Slid tile" message
- a wall bump that doesn't change the move count
- pressing Enter on an empty line with no crash
- switching size with 5

➤ /export

└ Conversation copied to clipboard

Chat History Link

https://drive.google.com/file/d/1E2-fKOcKtN_63OZ94q-h-jlpUEETeO/view?usp=drive_link